

Question Bank — Week 3

Recursion · data structures and classes in Python (Lecture 03, Lab 03) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Answers are in a separate file: [week03-answers.md](#). Levels: **[what]** recall · **[how]** apply · **[why]** explain and justify.

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W3-M01 – W3-M26
B	Short answer and essay	W3-E1 – W3-E6
C	Recurrences — write and solve	W3-R1 – W3-R5
D	Trace the code and the call stack	W3-T1 – W3-T8
E	Find and fix the bug	W3-B1 – W3-B4
F	Write the code — checked by pytest	W3-C1 – W3-C5

Part A — Multiple choice

W3-M01 [what] Every correct recursive function needs:

- **a)** a base case, and a recursive case that moves closer to the base case
- **b)** a loop and a return statement
- **c)** at least two recursive calls
- **d)** a global variable to count the calls

W3-M02 [how] With `def factorial(n): return 1 if n == 0 else n * factorial(n - 1)`, what does `factorial(-1)` do?

- **a)** returns 1
- **b)** returns -1
- **c)** raises `ValueError`
- **d)** raises `RecursionError`

W3-M03 [what] By default, CPython stops a recursion at a depth of about:

- **a)** 100
- **b)** 1,000
- **c)** 100,000
- **d)** there is no limit

W3-M04 [how] How much stack space does `factorial(n)` use?

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** $O(n!)$

W3-M05 [how] $T(n) = T(n/2) + c$ solves to:

- **a)** $O(\log n)$
- **b)** $O(n)$
- **c)** $O(n \log n)$
- **d)** $O(1)$

W3-M06 [how] $T(n) = 2T(n-1) + c$ solves to:

- **a)** $O(n)$
- **b)** $O(n^2)$
- **c)** $O(n \log n)$
- **d)** $O(2^n)$

W3-M07 [how] $T(n) = T(n-1) + cn$ solves to:

- **a)** $O(n)$
- **b)** $O(n^2)$
- **c)** $O(2^n)$
- **d)** $O(n \log n)$

W3-M08 [how] How many moves does Towers of Hanoi need for 5 disks?

- **a)** 25
- **b)** 32
- **c)** 31
- **d)** 10

W3-M09 [how] How many subsets does a set of 4 items have (including the empty set and the whole set)?

- **a)** 8
- **b)** 16
- **c)** 24
- **d)** 15

W3-M10 [how] How many orderings (permutations) do 4 distinct items have?

- **a)** 24
- **b)** 16
- **c)** 12
- **d)** 4

W3-M11 [how] How many calls in total does the naive `fib(5)` make (counting the first)?

- **a)** 5
- **b)** 8

- **c)** 9
- **d)** 15

W3-M12 [why] Fast power written as `half = power(x, n // 2) * power(x, n // 2)` (then $\times x$ if n is odd). Its cost is:

- **a)** $O(\log n)$ — it halves n
- **b)** $O(1)$
- **c)** $O(n)$ — two calls on $n/2$ undo the saving
- **d)** $O(n^2)$

W3-M13 [why] `list_max(values)` recurses on `values[1:]`. Its running time is:

- **a)** $O(n)$
- **b)** $O(n^2)$ — every slice copies the rest
- **c)** $O(\log n)$
- **d)** $O(n \log n)$

W3-M14 [what] Memoisation means:

- **a)** storing the answer to each subproblem the first time and looking it up after
- **b)** replacing recursion with a loop
- **c)** raising the recursion limit
- **d)** printing each call to debug it

W3-M15 [why] `hanoi(20)` makes over a million calls. At most how many frames are on the call stack at the same time?

- **a)** about 1,000,000
- **b)** 2^{20}
- **c)** 1,000 — the recursion limit
- **d)** about 20

W3-M16 [how] What does `f(3)` print?

```
def f(n):  
    if n == 0:  
        return  
    print(n, end=" ")  
    f(n - 1)  
    print(n, end=" ")
```

- **a)** 3 2 1
- **b)** 1 2 3 3 2 1
- **c)** 3 2 1 1 2 3
- **d)** 3 3 2 2 1 1

W3-M17 [why] What should `subsets([])` return?

- **a)** `[[[]]`
- **b)** `[]`
- **c)** `None`

- **d)** [None]

W3-M18 [what] Tail-call optimisation in Python:

- **a)** is done automatically for every tail call
- **b)** is not done: every recursive call costs a stack frame
- **c)** is done only when `sys.setrecursionlimit` is called
- **d)** turns recursion into a loop at compile time

W3-M19 [how] `marks = [3, 1, 2]; result = marks.sort()`. What is result?

- **a)** [1, 2, 3]
- **b)** [3, 1, 2]
- **c)** an error
- **d)** None

W3-M20 [what] What is `type({})`?

- **a)** `<class 'set'>`
- **b)** `<class 'list'>`
- **c)** `<class 'dict'>`
- **d)** `<class 'tuple'>`

W3-M21 [what] The average cost of `x in s` for a Python set of n items is:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** $O(n^2)$

W3-M22 [why] What is wrong with this class?

```
class Dog:
    tricks = []
    def __init__(self, name):
        self.name = name
    def add_trick(self, trick):
        self.tricks.append(trick)
```

- **a)** `__init__` must return `self`
- **b)** `add_trick` needs a return
- **c)** `name` should be a class variable
- **d)** every `Dog` shares one `tricks` list

W3-M23 [what] Calling a function that contains `yield`:

- **a)** runs the body and returns the first yielded value
- **b)** returns a generator object without running the body yet
- **c)** runs the whole body and returns a list
- **d)** raises `StopIteration`

W3-M24 [why] Why should a queue not be a Python list used with `pop(0)`?

- **a)** `pop(0)` raises an error on lists
- **b)** lists cannot hold more than 1,000 items
- **c)** `pop(0)` shifts every remaining item, $O(n)$ per dequeue
- **d)** `pop(0)` removes the last item, not the first

W3-M25 [how] What is counts after this?

```
counts = {}  
for w in "a b a".split():  
    counts[w] = counts.get(w, 0) + 1
```

- **a)** `{'a': 1, 'b': 1}`
- **b)** `{'a': 2, 'b': 1}`
- **c)** a `KeyError`
- **d)** `{'a': 2}`

W3-M26 [how] The recurrence of naive Fibonacci, $T(n) = T(n - 1) + T(n - 2) + c$, grows like:

- **a)** $O(n)$
 - **b)** $O(n^2)$
 - **c)** $O(n \log n)$
 - **d)** $O(\varphi^n)$ with $\varphi \approx 1.618$
-

Part B — Short answer and essay

W3-E1 [how] (4 marks) Draw the call stack at every step of `factorial(3)`, from the first call until the final value is returned. Label winding, the base case, and unwinding, and show the value each frame returns.

W3-E2 [why] (4 marks) Explain why the naive recursive Fibonacci takes exponential time, using a recursion tree. Show how memoisation fixes it, and state the new time and space costs.

W3-E3 [why] (3 marks) When should you prefer recursion, and when a loop? Include two concerns that are specific to Python.

W3-E4 [why] (4 marks) Describe the recursive strategy for Towers of Hanoi. Write its recurrence, solve it, and explain why no algorithm can use fewer moves.

W3-E5 [why] (3 marks) Explain “trust the recursion” and how it relates to mathematical induction.

W3-E6 [why] (4 marks) For each job, choose `list`, `tuple`, `set` or `dict` and justify by cost: (a) the marks of a class, in the order entered; (b) checking whether a student ID has already been registered; (c) looking up a student’s name by ID; (d) one student’s (ID, name, mark), used as a key elsewhere.

Part C — Recurrences

For each function write the recurrence for its running time $T(n)$ (and the base case), then solve it.

W3-R1 [how] n is the number of digits of the input.

```
def sum_digits(x):  
    if x < 10:  
        return x  
    return x % 10 + sum_digits(x // 10)
```

W3-R2 [why]

```
def mystery(n):  
    if n <= 1:  
        return 1  
    return mystery(n - 1) + mystery(n - 1)
```

What does `mystery(n)` return? Rewrite it to make it $O(n)$ — and then $O(1)$.

W3-R3 [why] $n = hi - lo$.

```
def range_sum(arr, lo, hi):  
    # sum of arr[lo .. hi-1]  
    if hi - lo == 0:  
        return 0  
    if hi - lo == 1:  
        return arr[lo]  
    mid = (lo + hi) // 2  
    return range_sum(arr, lo, mid) + range_sum(arr, mid, hi)
```

What is its stack depth?

W3-R4 [how] $n = \text{len}(\text{text})$.

```
def reverse(text):  
    if text == "":  
        return ""  
    return reverse(text[1:]) + text[0]
```

W3-R5 [why] Prove by induction that the Hanoi recurrence $T(1) = 1$, $T(n) = 2T(n - 1) + 1$ has the solution $T(n) = 2^n - 1$.

Part D — Trace the code and the call stack

W3-T1 [how] What is printed?

```
def g(n):  
    if n < 10:
```

```

        return n
    return g(n // 10) + n % 10

print(g(4096), g(7))

```

W3-T2 [how] What is printed?

```

def h(n):
    if n < 2:
        return str(n)
    return h(n // 2) + str(n % 2)

print(h(13), h(1), h(8))

```

W3-T3 [how] List every call made by `gcd(48, 18)`, in order, with the value each returns. What is the maximum stack depth?

```

def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)

```

W3-T4 [how] What is printed?

```

calls = 0

def fib(n):
    global calls
    calls += 1
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(6), calls)

```

W3-T5 [how] `hanoi(n, source, target, spare)` moves $n - 1$ disks from source to spare, moves one disk from source to target, then moves $n - 1$ disks from spare to target. List the moves of `hanoi(2, "A", "C", "B")`, and give the **4th** move of `hanoi(3, "A", "C", "B")`.

W3-T6 [how] What is printed?

```

def count_down_up(n):
    if n == 0:
        return [0]
    return [n] + count_down_up(n - 1) + [n]

print(count_down_up(2), len(count_down_up(5)))

```

W3-T7 [how] What is printed?

```
words = "the cat the hat".split()
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1
print(counts)
print(sorted(counts, key=lambda w: -counts[w])[0], {len(w) for w in words})
```

W3-T8 [how] What is printed?

```
class Countdown:
    def __init__(self, n):
        self.n = n

    def __iter__(self):
        k = self.n
        while k > 0:
            yield k
            k -= 1

c = Countdown(3)
print(list(c), sum(c), len(list(c)))
```

Part E — Find and fix the bug

W3-B1 [why] Correct results, but far too slow. Why, and how do you fix it?

```
def power(x, n):
    if n == 0:
        return 1
    if n % 2 == 0:
        return power(x, n // 2) * power(x, n // 2)
    return x * power(x, n - 1)
```

W3-B2 [how] Should return every subset of items.

```
def subsets(items):
    if not items:
        return []
    rest = subsets(items[1:])
    return rest + [[items[0]] + s for s in rest]
```

W3-B3 [how] Should flatten nested lists: `flatten([1, ["ab", [2]])` is `[1, "ab", 2]`.

```
def flatten(nested):
    result = []
    for item in nested:
```

```
    if hasattr(item, "__iter__"):
        result += flatten(item)
    else:
        result.append(item)
return result
```

W3-B4 [how] Should return the sum of a list.

```
def total(values, i=0):
    if i == len(values):
        return 0
    values[i] + total(values, i + 1)
```

Part F — Write the code

In `practice/week03.py`; check with `pytest tests/test_practice_week03.py -v`. Every function must be **recursive**.

W3-C1 [how] `array_sum(arr, i=0)` — sum of a course Array from index `i`, without slicing.

W3-C2 [how] `count_char(text, ch)` — occurrences of one character.

W3-C3 [how] `is_sorted_rec(arr, i=0)` — is the Array non-decreasing from `i`?

W3-C4 [why] `no_consecutive_ones(n)` — every binary string of length `n` with no "11". How many are there for `n = 1, 2, 3, 4, 5`, and why?

W3-C5 [how] `pascal_row(n)` — row `n` of Pascal's triangle, built from row `n - 1`.